

Tercer Trabajo Práctico

Alumno: Lautaro Leal Del Prete
Legajo: UIA 70 318

Introducción

Este trabajo práctico tiene como objetivo implementar tests de integración utilizando Postman y Newman. A diferencia de los TPs anteriores, donde se realizaron pruebas unitarias sobre funciones aisladas, aquí se prueban endpoints HTTP reales de una API externa, verificando que los métodos REST (GET, POST, PUT, PATCH y DELETE) respondan correctamente según los contratos definidos.

Se utilizó la API pública **JSONPlaceholder** (<https://jsonplaceholder.typicode.com>), diseñada específicamente para testing y prototipado. Las pruebas se organizaron en una colección de Postman con variables de entorno, y se ejecutaron de forma automatizada desde la línea de comandos mediante Newman, el runner CLI de Postman.

Herramientas Utilizadas

Para llevar a cabo las pruebas de integración se utilizaron las siguientes herramientas:

- **Postman**: plataforma para diseñar, ejecutar y documentar colecciones de requests HTTP con scripts de test en JavaScript.
- **Newman**: runner CLI oficial de Postman que permite ejecutar colecciones desde la terminal e integrarse con pipelines de CI/CD.
- **newman-reporter-htmlextra**: plugin que genera reportes HTML detallados con visualización de resultados por request y assertion.
- **Node.js / npm**: entorno de ejecución y gestor de paquetes para instalar Newman y ejecutar el proyecto.

Colección de Tests Implementada

Se creó una colección llamada **JSONPlaceholder TP3** con 5 requests y 8 assertions en total, cubriendo los cinco métodos HTTP principales sobre el recurso */posts*. Todos los tests pasaron sin errores (0 fallos).

Request	Método	Endpoint	Tests
GET Post	GET	{{url}}/posts/{{postId}}	2
POST Post	POST	{{url}}/posts	2
PUT Post	PUT	{{url}}/posts/{{postId}}	1

PATCH Post	PATCH	{{url}}/posts/{{postId}}	2
DELETE Post	DELETE	{{url}}/posts/{{postId}}	1

Configuración del Entorno (Environment)

Se configuró un entorno en Postman llamado **JSONPlaceholder_Env** con dos variables que parametrizan las requests, evitando hardcodear valores en la colección:

Variable	Valor	Descripción
url	https://jsonplaceholder.typicode.com	URL base de la API
postId	1	ID del post usado en las requests

Pruebas de Integración

Se realizaron 8 assertions distribuidas en los 5 requests de la colección. A continuación se describen las pruebas implementadas:

- **[GET]** *Status code is 200 (GET)* — Verifica que la respuesta del GET sea 200 OK.
- **[GET]** *Response has userId* — Verifica que el JSON de respuesta contenga la propiedad 'userId'.
- **[POST]** *Status code is 201 (POST)* — Verifica que la creación retorne 201 Created.
- **[POST]** *Response contains id 101* — Verifica que el nuevo recurso tenga id = 101 (comportamiento de JSONPlaceholder).
- **[PUT]** *Status code is 200 (PUT)* — Verifica que la actualización completa retorne 200 OK.
- **[PATCH]** *Status code is 200 (PATCH)* — Verifica que la actualización parcial retorne 200 OK.
- **[PATCH]** *Title was updated* — Verifica que el título en la respuesta sea el valor enviado ('patched title').
- **[DELETE]** *Status code is 200 (DELETE)* — Verifica que la eliminación retorne 200 OK.

Código Fuente

JSONPlaceholder_Collection.json — Colección de Postman exportada

```
{
  "info": {
    "name": "JSONPlaceholder TP3",
    "description": "TP3 Test de Integracion - GET, POST, PUT, PATCH, DELETE",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "GET Post",
      "event": [{ "listen": "test", "script": { "exec": [
        "pm.test(\"Status code is 200 (GET)\", function () {"
```

```

        "    pm.response.to.have.status(200);",
        "});",
        "pm.test(\"Response has userId\", function () {",
        "    var jsonData = pm.response.json();",
        "    pm.expect(jsonData).to.have.property('userId');",
        "});",
    ]}],
    "request": { "method": "GET", "url": "{{url}}/posts/{{postId}}" },
},
{
    "name": "POST Post",
    "event": [{ "listen": "test", "script": { "exec": [
        "pm.test(\"Status code is 201 (POST)\", function () {",
        "    pm.response.to.have.status(201);",
        "});",
        "pm.test(\"Response contains id 101\", function () {",
        "    var jsonData = pm.response.json();",
        "    pm.expect(jsonData.id).to.eql(101);",
        "});",
    ]}],
    "request": { "method": "POST", "body": { "raw": "{title,body,userId}" },
        "url": "{{url}}/posts" },
},
{
    "name": "PUT Post",
    "event": [{ "listen": "test", "script": { "exec": [
        "pm.test(\"Status code is 200 (PUT)\", function () {",
        "    pm.response.to.have.status(200);",
        "});",
    ]}],
    "request": { "method": "PUT", "url": "{{url}}/posts/{{postId}}" },
},
{
    "name": "PATCH Post",
    "event": [{ "listen": "test", "script": { "exec": [
        "pm.test(\"Status code is 200 (PATCH)\", function () {",
        "    pm.response.to.have.status(200);",
        "});",
        "pm.test(\"Title was updated\", function () {",
        "    var jsonData = pm.response.json();",
        "    pm.expect(jsonData.title).to.eql(\"patched title\");",
        "});",
    ]}],
    "request": { "method": "PATCH", "body": { "raw": "{\\\"title\\\": \\\"patched title\\\"}"
},
        "url": "{{url}}/posts/{{postId}}" },
},
{
    "name": "DELETE Post",

```

```

    "event": [{ "listen": "test", "script": { "exec": [
      "pm.test(\"Status code is 200 (DELETE)\", function () {",
      "    pm.response.to.have.status(200);",
      "});",
    ]}],
    "request": { "method": "DELETE", "url": "{{url}}/posts/{{postId}}" }
  }
]
}

```

JSONPlaceholder_Environment.json — Entorno de Postman exportado

```

{
  "id": "e2a0f8b1-1b9a-4c91-95e2-0402b4d18fa1",
  "name": "JSONPlaceholder_Env",
  "values": [
    { "key": "url", "value": "https://jsonplaceholder.typicode.com", "enabled": true },
    { "key": "postId", "value": "1", "enabled": true }
  ],
  "_postman_variable_scope": "environment",
  "_postman_exported_at": "2026-05-11T00:00:00.000Z",
  "_postman_exported_using": "Postman/10.0.0"
}

```

package.json — Configuración del proyecto y script de ejecución

```

{
  "name": "3er-tp",
  "version": "1.0.0",
  "scripts": {
    "test": "newman run JSONPlaceholder_Collection.json
      -e JSONPlaceholder_Environment.json -r cli,htmlextra"
  },
  "dependencies": {
    "newman": "^6.2.2",
    "newman-reporter-htmlextra": "^1.23.1"
  }
}

```

Ejecución con Newman

Se ejecutó la colección desde la terminal con el comando **npm run test**, que internamente invoca Newman con el reporte *cli* y *htmlextra*. Todos los tests pasaron correctamente:

Métrica	Ejecutados	Fallidos
Iterations	1	0
Requests	5	0
Test-scripts	5	0
Prerequisite-scripts	0	0
Assertions	8	0

Total run duration: 1275ms | Total data received: 667B (approx) | Average response time: 170ms [min: 150ms, max: 179ms, s.d.: 10ms]

Captura de pantalla del resultado en terminal:

```
$ npm run test

> 3er-tp@1.0.0 test
> newman run JSONPlaceholder_Collection.json -e JSONPlaceholder_Environment.json -r cli,htmlextra

newman

JSONPlaceholder TP3

→ GET Post
GET https://jsonplaceholder.typicode.com/posts/1 [200 OK, 1.44kB, 150ms]
✓ Status code is 200 (GET)
✓ Response has userId

→ POST Post
POST https://jsonplaceholder.typicode.com/posts [201 Created, 1.29kB, 175ms]
✓ Status code is 201 (POST)
✓ Response contains id 101

→ PUT Post
PUT https://jsonplaceholder.typicode.com/posts/1 [200 OK, 1.2kB, 176ms]
✓ Status code is 200 (PUT)

→ PATCH Post
PATCH https://jsonplaceholder.typicode.com/posts/1 [200 OK, 1.37kB, 179ms]
✓ Status code is 200 (PATCH)
✓ Title was updated

→ DELETE Post
DELETE https://jsonplaceholder.typicode.com/posts/1 [200 OK, 1.11kB, 172ms]
✓ Status code is 200 (DELETE)
```

	executed	failed
iterations	1	0
requests	5	0
test-scripts	5	0
prerequest-scripts	0	0
assertions	8	0
total run duration: 1275ms		
total data received: 667B (approx)		
average response time: 170ms [min: 150ms, max: 179ms, s.d.: 10ms]		

Figura 1 — Resultado de `npm run test` mostrando 5 requests y 8 assertions pasadas.

Conclusiones

El trabajo práctico permitió aplicar los conceptos de testing de integración sobre una API REST real. A diferencia de las pruebas unitarias de los TPs anteriores, aquí se verifican contratos entre sistemas: que los endpoints respondan con los códigos HTTP correctos y que el cuerpo de las respuestas tenga la estructura esperada.

El uso de variables de entorno en Postman facilita la reutilización de la colección en distintos ambientes (desarrollo, staging, producción) sin modificar los requests. Newman, por su parte, permite integrar estas pruebas en flujos de integración continua, cerrando el ciclo de

automatización del testing.